How nixnan works: from load to the first HMMA report

SC26 tutorial walk-through, using the reaction–diffusion demos

Ganesh Gopalakrishnan

2026-10-19

Roadmap

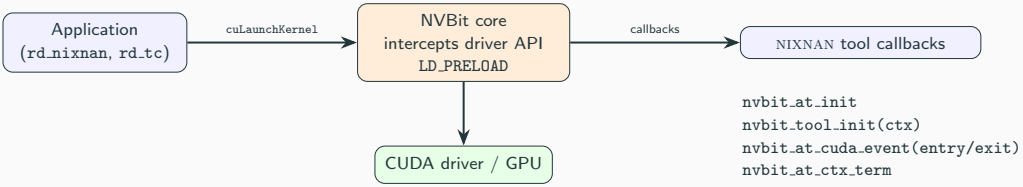
1. Setup, and the one environment trap
2. Callback architecture (NVBit $\leftrightarrow$ NIXNAN)
3. Lifecycle of the tool
4. First kernel launch: how instrumentation happens
5. Compact encoding: `instructions.json` $\rightarrow$ `reginfo` word
6. The two injected calls (`IPOINT_AFTER`/`BEFORE`)
7. Device side: classify, OR, count, push
8. GPU$\rightarrow$CPU channel, doorbell, receiver thread, sentinel
9. End-of-run report
10. **Part A:** an ordinary SASS instruction (`HFMA2` in `rd_nixnan`)
11. **Part B:** `HMMA.16816.F16` in `rd_tc`

Setup, and the trap

```
env -u LD_LIBRARY_PATH \
  LINE_INFO=1 PRINT_ILL_INSTR=1 TOOL_VERBOSE=1 \
  LOGFILE=latex/data/partA-scalar.log \
  LD_PRELOAD=~/repos/nixnan/nixnan.so ./rd_nixnan
```

- `LD_PRELOAD` loads NIXNAN (built on NVBit 1.8) before the CUDA runtime.
- On this machine, with my usual `LD_LIBRARY_PATH` (a missing `cuda-13.2/lib64` plus a dyninst lib dir) NIXNAN printed its banner and *nothing else*. With `env -u LD_LIBRARY_PATH` the same binary reports. (Observed this session; I did not isolate which entry is at fault.)
- An earlier driver-mismatch (kernel module 595.84 vs. library 595.99.02) was cured by a reboot.
- Branch `print_histo_instrn`, commit `ce35185`.

Callback architecture



Lifecycle of the tool (nixnan.cu)

`nvbit_at_init` reads env vars via `GET_VAR_*` (`INSTR_BEGIN/END`, `TOOL_VERBOSE`, `PRINT_ILL_INSTR`, `SAMPLING`, `LINE_INFO`, `LOGFILE`, `MAX_ERRORS`, white/blacklist...); sets `CUDA_MANAGED_FORCE_DEVICE_ALLOC=1`; prints the banner.

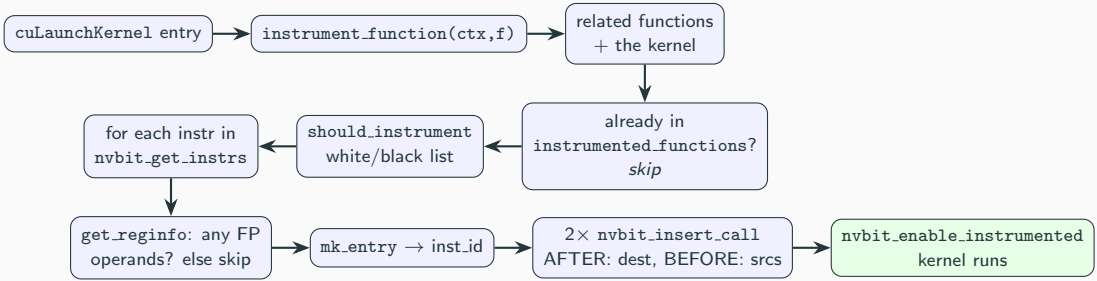
`nvbit_tool_init` "Initializing GPU context...": builds `recorder(TABLE_SIZE)`, `TABLE_SIZE` = 2^{17} entries; `channel_host.init(0, sizeof(exception_info), &channel_dev, NULL)`; starts `recv_thread`.

`nvbit_at_cuda_event`, **entry** for the launch APIs: `recv_thread_receiving = true`; `instrument_function`; prints "running kernel [...]" the first time; `nvbit_enable_instrumented(..., sampling>0 ? count%sampling==0 : true)`.

`nvbit_at_cuda_event`, **exit** `cudaDeviceSynchronize()`, check `cudaGetLastError()`.

`nvbit_at_ctx_term` stop and `join` the receiver, `recorder->end()`, print the report.

First launch of a kernel: what happens



`mk_entry` → `inst_id`
Host table, one row per instrumented instruction: SASS text, file:line, function, operand types. `inst_id` = row number (0, 1, 2, . . .), passed to the GPU as the instruction's only identity; the host looks the row up when printing.

2× call, HMMA example
`HMMA.16816.F16 R22, R16, R22, R8`
BEFORE reads in: A R16–R19, B R22–R23, C R8–R9
AFTER reads out: D R22–R23
(R22 is both B and D)

`nvbit_enable_instrumented`
NVBit keeps two copies of each function: the *original* and the *instrumented* one (with the injected calls). Each launch selects which runs: `true` → instrumented, `false` → original.

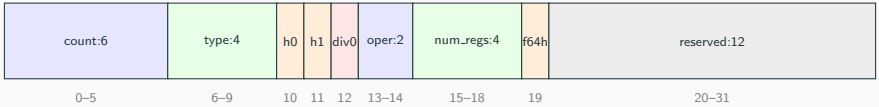
Table-driven: `instructions.json`

The opcode string is matched against keys (each tried as a regex with `regex_match`); the entry lists operand groups in SASS operand order.

```
"HMMA.16816.F16": {"registers": [
   {"type":"f16","count":4},   // operand 0  D (destination)
   {"type":"f16","count":8},   // operand 1  A
   {"type":"f16","count":4},   // operand 2  B
   {"type":"f16","count":4} ]} // operand 3  C
"(FFMA(?:\\.FTZ)?(?:\\.(RP|RM|RZ))?(?:\\.SAT)?)": ...   // regex key
```

- `count` = number of *values*; registers needed: `get_num_regs`: f16/bf16 $\lceil n/2 \rceil$, f32 n, f64 $2n$.
- Optional `"div0"` flag asks to flag zeros too.
- The JSON is embedded in the binary as a byte array (`instructions.h`) and parsed once (nlohmann).
- Opcodes with no entry (e.g. `F2FP.PACK`: grep count 0 in the JSON) are *not instrumented*.

One 32-bit word per operand: `reginfo`



Bit positions assume GCC's LSB-first bit-field allocation (my reading; not dumped from a binary).

- `static_assert(sizeof(reginfo)==4)` so it travels as *one* 32-bit call argument (`nvbit_add_call_arg_const_val32`).
- `type`: FP16=0, BF16=1, FP32=2, FP64=3. `operand` is 2 bits: at most 4 operands (`OPERANDS=4`).
- `count` and `num_regs` are signed `int` bit-fields (6 and 4 bits), so `num_regs` is meaningful only up to 7 though `get_reginfo` asserts < 16.
- `half_h0/h1`: which half of a packed pair to inspect.

Turning an operand into register reads: `get_regs`

Operand kind	What nixnan does
`REG` / `UREG`	one lambda per register: `nvbit_add_call_arg_reg_val` / `ureg_val(instr, clamp(reg+i))`
`CBANK`	lambda: `nvbit_add_call_arg_cbank_val(instr, id, imm_offset+4i)`
`IMM_DOUBLE`	checked on the host; prints a warning if NaN/Inf/subnormal; no runtime arg
`IMM_UINT64`	ignored (`num_regs=0`)
`GENERIC`	text-matched for NAN/INF, warning only

The lambdas are *deferred*: they are stored in the `reginsertion` vectors and executed only *after* the call's fixed arguments have been added, so argument order is correct. `f64high` opcodes (suffix `64H`) start one register lower.

Book-keeping on the host: `recorder::mk_entry`

Each instrumented instruction gets a dense `inst_id` (the table index) and a `host_entry`:

- SASS string (`instr->getSass()`), function name (`cut_kernel_name`), up to 4 operand types.
- Source location, if `LINE_INFO`: `nvbit_get_line_info(ctx,f,offset,...)` under a temporary `SIGSEGV` handler (prints "try LINE_INFO=0" and exits if NVBit crashes reading debug info).

Device counters: `device_errors` = `cudaMalloc`'d array of $2^{17} \cdot 2^{7}$ `uint32` zero-initialised; index

$$\texttt{id} \ll 7 \mid \texttt{op} \ll 5 \mid \texttt{exce} \quad (\text{SH_AMT} = \text{EXCEBITS} + \text{OPBITS} = 5 + 2).$$

The ID is the only thing the GPU needs to send back; strings never leave the host.

Two injected calls per instruction

IPOINT_AFTER: operand **0** (the destination)

```
nvbit_insert_call(instr,"nixnan_check_regs",
                  IPOINT_AFTER);
guard_pred_val
const64 recorder->device_recorder
const32 inst_id
const64 &channel_dev
const32 1 + rfuns.size()   // arg_count
const32 reginfo(op 0)
reg values via lambdas...
```

IPOINT_BEFORE: operands **1...n** (sources)

```
nvbit_insert_call(instr,"nixnan_check_regs",
                  IPOINT_BEFORE);
guard_pred_val
const64 device_recorder
const32 inst_id
const64 &channel_dev
const32 sum(num_regs) + (#ops-1)
{const32 reginfo(op i); regs...}*
```

Destination is read *after* the instruction (the result), sources *before* (so overwritten registers, e.g. R26 as both C and D, are seen with their old value). `nvbit_add_call_arg_guard_pred_val` passes the guard predicate: predicated-off threads do nothing.

nixnan_check_regs on the GPU (inject_funs.cu)

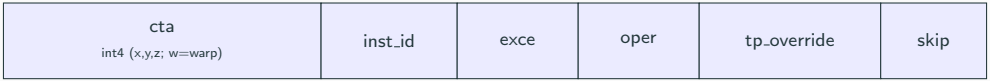
1. `if (!pred) return;` active mask by `__ballot_sync`; `first_laneid = __ffs(mask)-1`.
2. `va_arg` loop: read a `reginfo`, then `count` values; classify by `type`:

FP16 `half2_classify(val,h0,h1,div0)`; consumes 2 values per register
BF16 `bf162_classify(...)`
FP32 `float_classify(val,div0)`
FP64 two words (low, high) → `double_classify`

3. Result bits OR-ed into `excep[operand]`: `E_NAN=1, E_INF=2, E_NINF=4, E_SUB=8, E_DIV0=16`.
4. Shuffle-OR across the 32 lanes (`__shfl_sync` loop) so every lane has the warp's verdict.
5. First active lane: for each operand with `excep!=0`: `old = recorder.record(id,excep,op)` = `atomicAdd(device_errors+idx,1)`.
6. **Only if** `old==0` (first time this (instr, operand, flag-set) is seen) push `exception_info` to the channel (twice, see later).

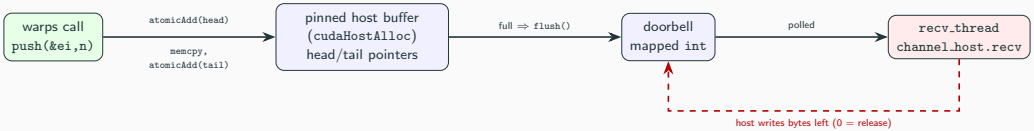
Repeats are therefore only *counted*, never transmitted: that is the "(N repeats)" in the final report.

The packet: exception_info



- Fixed-size struct; `CHANNEL_SIZE = sizeof(exception_info)`.
- `tp_override = UNKNOWN` (127) in normal use ⇒ the receiver looks the type up from the recorder (`get_type(id,operand)`).
- The receiver turns `exce` bits into “NaN”, “infinity”, “-infinity”, “subnormal”, “div0” and prints with the stored SASS, function and `path:line`.

GPU → CPU channel (NVBit `channel.hpp`)



push: reserve space with `atomicAdd` on the head; if it runs past `buff_end`, the first warp to notice waits for tail==its pointer and calls `flush()`; others spin until head is reset.

flush: `__threadfence_system()`; `*doorbell = nbytes`; **spin until host sets doorbell to 0**; reset tail then head. Corner-cases to avoid deadlocks are incorporated.

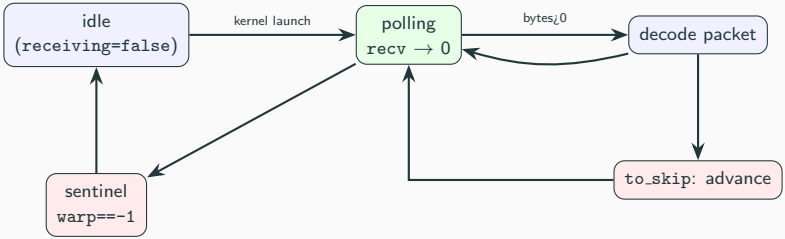
Receiver: `recv` and the thread loop

`ChannelHost::recv(buf, max)`: read `*doorbell`; 0 ⇒ return 0. Else copy $\min(n, \texttt{max})$ bytes from the pinned buffer (read head advances); write `*doorbell = bytes_left`. When it reaches 0 the GPU resumes. Since `max = sizeof(exception_info)`, the host drains *one packet at a time*, each time re-arming the doorbell with the bytes left.

`recv_thread_fun` (`nixnan.cu`):

- busy loop while `recv_thread_started`; works only if `recv_thread_receiving` (set at kernel launch).
- `to_skip()` packets are stepped over (`continue`).
- `warp()==-1` ⇒ `recv_thread_receiving=false` (the sentinel).
- otherwise decode and `nnout_line()` (mutex-guarded, flushed; avoids torn lines).
- `MAX_ERRORS` reached ⇒ `nnterminate`.

Receiver state machine



Why `skip`? The channel buffer is exactly one packet (`CHANNEL_SIZE`), and `push` only flushes when the *next* packet does not fit. So a lone real packet would sit unflushed. Each first occurrence is therefore pushed twice: the real one (`skip=0`) and a dummy (`skip=1`) whose arrival forces the real one out; the receiver steps over the dummy. (My reading of the code; no comment in the repository states the intent. It is consistent with all observed runs, see the sentinel slide.)

The “one last message”: sentinel flush — design vs. this commit

```
__global__ void flush_channel() {
  exception_info ei(int4{-1,-1,-1,-1}, 0,0,0,0);   // warp() == -1
  channel_dev.push(&ei, sizeof(exception_info));
  channel_dev.flush();
}
```

- **Design:** after each kernel, launch `flush_channel<<<1,1>>>`, so (a) any partially filled buffer is flushed and (b) the receiver sees `warp==-1` and clears `recv_thread_receiving`; the exit callback then waits `while(recv_thread_receiving) sched_yield();` so no report is lost and the doorbell spin cannot hang on a half-drained buffer.
- **Status in commit** `ce35185`**:** the launch of `flush_channel`, the `skip_flag` toggling and the wait loop in `nvbit_at_cuda_event` are **commented out**. The receiver still understands the sentinel, but nothing sends it. The exit callback now only does `cudaDeviceSynchronize()` and an error check.
- **Checked:** in the three `rd_tc` runs the live `error [...]` lines match the final report per flag (e.g. fp16acc32: NaN 50 live $= 16$ f16 $+ 34$ f32; Inf $12 = 8 + 4$; bf16acc32: NaN $66 = 24 + 42$, Inf $36 = 16 + 20$). No first occurrence was lost in these runs. This does not prove the sentinel is unnecessary: the dummy `skip` packet may be what makes it work, and the last dummy always stays in the buffer.

End of run

- `nvbit_at_ctx_term`: `recv_thread_started=false`; `join`.
- `recorder->end()`: `cudaMemcpy` `device_errors` → `host_errors`.
- For every (inst, operand, flag-set) with count > 0: bucket by (type, is_mem) and by flag (NaN, Inf, -Inf, Sub, Div0): *distinct locations* = number of buckets hit, *repeats* = total − distinct.
- Printed as "`---------- nixnan Report ----------`" with FP16/BF16/FP32/FP64 sections (and memory sections).

Note the live lines show each (instruction, operand, flag-set) *once*; the table gives how many times it recurred.

Part A — an ordinary SASS instruction: HFMA2 (rd_step_fp16)

Real SASS from `cuobjdump -sass rd_nixnan`:

```
/*0130*/ HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0 ;
```

NIXNAN (`TOOL_VERBOSE=1`) logged:

```
Instrumenting instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].H0_H0, R9.H0_H0 ;
Instrumenting instruction with 2 registers          <- AFTER: reginfo + 1 reg
Instrumenting operand 0. div0: 0, regs: 1, count: 2
Instrumenting instruction with 6 registers          <- BEFORE: 3 reginfo + 3 regs
Instrumenting operand 1. div0: 0, regs: 1
Instrumenting operand 2. div0: 0, regs: 1
Instrumenting operand 3. div0: 0, regs: 1
```

- Four groups: D=R0 (1 reg, 2 halves), A=R0, B=**constant bank** c[0x0][0x170], C=R9. Operand 2 is read with `nvbit_add_call_arg_cbank_val`.
- AFTER: `arg_count` = 1 reginfo + 1 reg = 2. BEFORE: 3 reginfo + 3 regs = 6.

Part A — what came back

Report lines for this instruction (first occurrences, real output):

```
error [infinity] detected in operand 1 of instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].H0_H0, R9.H0_H0 ;
     in function rd_step_fp16 of type f16
error [infinity] detected in operand 0 of instruction HFMA2 R0, R0.H0_H0, c[0x0] [0x170].H0_H0, R9.H0_H0 ;
     in function rd_step_fp16 of type f16
```

- Operand 1 inf (a *source*, checked BEFORE): the previous step's overflow arrived in the input.
- Operand 0 inf (the *destination*, checked AFTER): the multiply-add itself produced/propagated it.
- No `at file:line`: `rd_nixnan` was built without `-lineinfo`, so `nvbit_get_line_info` returns nothing and the location is omitted.
- The type shown, `f16`, comes from the stored `opertypes[op]`, not from the packet (`tp_override=UNKNOWN`).

Also visible in the same log: `STG.E.U16` gets "Unsupported store instruction" (memory instrumentation is off unless `INSTR_MEM=1`).

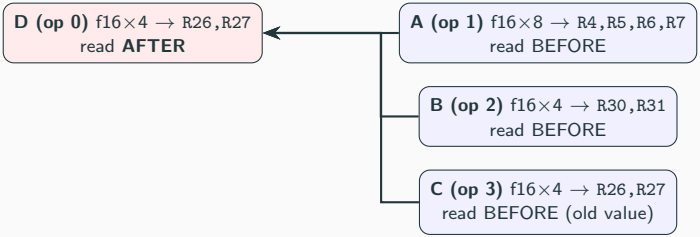
Part B — HMMA.16816.F16: one warp instruction, four register groups

```
/*0840*/ HMMA.16816.F16 R26, R4, R30, R26 ;      // cuobjdump -sass rd_tc
```

```
Instrumenting instruction with 3 registers      <- AFTER:  1 reginfo + 2 regs
Instrumenting operand 0. div0: 0, regs: 2, count: 4
Instrumenting instruction with 11 registers     <- BEFORE: 3 reginfo + 8 regs
Instrumenting operand 1. div0: 0, regs: 4
Instrumenting operand 2. div0: 0, regs: 2
Instrumenting operand 3. div0: 0, regs: 2
```

(Verbatim lines from the log for other HMMAs of the same kernel; they are identical for every `HMMA.16816.F16`.)

Part B — which registers are picked up



$m16n8k16$ half-precision: per thread the A fragment is 8 halves (4 registers), B 4 halves (2), C/D 4 halves (2). Starting registers come from the SASS text (R4, R30, R26); NIXNAN takes `reg_start + i` for $i <$ `num_regs`. Consequently the 8 BEFORE registers plus 3 reginfo words = 11 call arguments; the AFTER call has 3. Every active lane of the warp supplies its own copy (the device function runs per thread, then ORs across lanes).

Part B — reginfo words for this HMMA (verified offline)

Using the layout above (count, type, h0, h1, operand, num_regs):

operand	count	num_regs	word
0 (D)	4	2	`0x00010C04`
1 (A)	8	4	`0x00022C08`
2 (B)	4	2	`0x00014C04`
3 (C)	4	2	`0x00016C04`

Verification: the words were produced by compiling the real `reginfo.cuh` with g++ and filling the fields as `get_reginfo` does (all fields zeroed first). They match my hand computation. They were *not* dumped from the running tool; the tool leaves the 12 reserved bits uninitialised, which the device code ignores. For `.F32`: D = `0x00020C84`, C = `0x00026C84`.

Limits worth knowing: `num_regs` has 4 signed bits; `get_reginfo` asserts `get_num_regs<16`.
HMMA.1684 TF32 and `.F32`/`.F32.BF16` variants have their own entries (f32 count 4 for D and C).

Part B — what nixnan reported (real output, `fp16acc16`)

```
error [infinity] detected in operand 0 of instruction HMMA.16816.F16 R26, R4, R30, R26 ; ...
error [infinity] detected in operand 0 of instruction HMMA.16816.F16 R4, R8.reuse, R4, R22 ; ...
error [infinity] detected in operand 3 of instruction HMMA.16816.F16 R6, R8, R6, R26 ; ...
  step 230: max|finite|=6.5280e+04  inf=120  nan=0          (program's own count)
error [NaN] detected in operand 0 of instruction HMMA.16816.F16 R22, R16, R22, R8 ; ...
```

- Operand 0 inf: the tensor core *produced* the overflow in its f16 accumulator (step 230).
- Next step: inf inputs meet the zeros of P: $0 \times \infty =$ NaN inside HMMA; all 4096 entries are NaN one step later.
- With `fp16acc32`/`bf16acc32` the first inf is born in `F2FP.PACK`/`F2FP.BF16.PACK`. **Verified:** `grep -c F2FP instructions.json` $= 0$ and the logs contain no `F2FP` line, so that conversion is not instrumented; NIXNAN first shows the inf as an operand of the *next* HMMA.
- `LINE_INFO=1` did not give `mma_sync` source lines in these runs (no `at file:line`).

Part B — histogram mode: how close do the HMMA operands get to overflow?

`HISTOGRAM=1 BIN_SPEC_FILE=spec.json`: the same injection machinery, but each operand's *exponent* is binned and every `count`-th hit in a requested range is reported. Spec: top binade only (`count` 64, f16 [15,15], bf16/f32 [127,127]).

mode	f16 top	bf16 top	f32 top
fp16acc16	3743	–	–
fp16acc32	2791	–	**0**
bf16acc32	–	2531	853

("bin reached threshold" lines.)

- fp16acc32: the f32 accumulator *never* reaches the top f32 binade, so the overflow is made by the FP32→FP16 conversion, matching the F2FP finding.
- bf16acc32: f32 accumulators do reach $[2^{127}, 2^{128})$, as the native max finite 3.39×10^{38} at step 1764 suggests.
- **Tool caveats seen:** histogram and exception lines interleave mid-line in the log; the run ends with `terminate called without an active exception` (abort at exit); wrapping the program in `timeout` made NIXNAN silent (use `NIXNAN_TIMEOUT`).

Summary

- Tool is a table: opcode → operand groups (type, count); everything else is generic.
- One 32-bit `reginfo` per operand; the GPU function is variadic.
- Two injected calls: destination AFTER, sources BEFORE.
- Warp-level OR, per-site atomic counters; only *first* occurrences cross the channel.
- Channel = pinned buffer + doorbell handshake; one polling receiver thread; sentinel in design but disabled in this commit.
- HMMA fits unchanged: it is just an opcode with four groups and 11 BEFORE arguments.

Files: `rd_nixnan/latex/data/` (raw logs), `nixnan-walkthru-report.pdf` (prose).